

Universidad Rafael Landívar

Lenguajes formales y autómatas

sección: 01

catedrático: Ing Julio Duarte

Proyecto02

Angel Samuel Pérez Cruz

1135323

Guatemala, 24 de octubre del 2025

Contenido

Introducción.....	3
Manual de uso	4
Gramática soportada	6
Arquitectura del sistema	8
Pruebas del Programa	9
Evidencia de resultados	11
Solución del error del archivo .dot	11
Consejos de uso	12
Mejoras futuras	12

Introducción

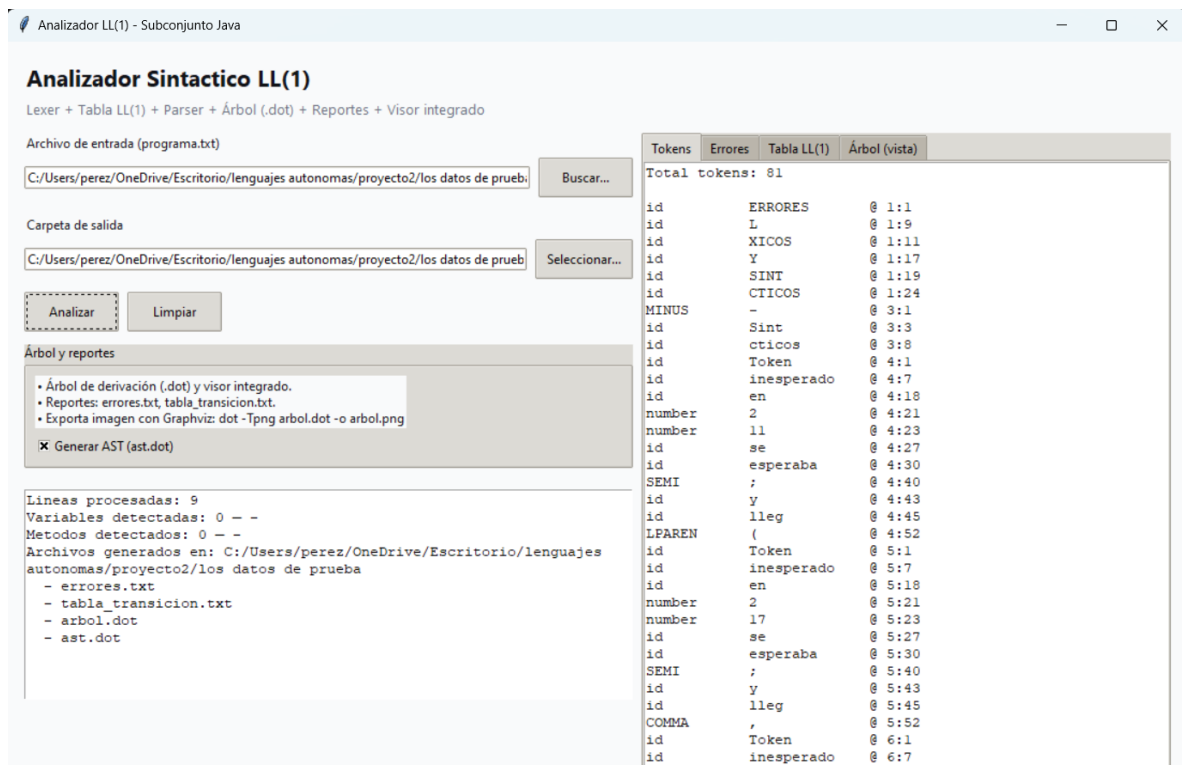
Este proyecto implementa un analizador sintáctico predictivo LL(1) para un subconjunto de Java, con:

- Léxico: tokenización con manejo de comentarios `//` y `/* ... */`.
- Sintaxis: cálculo de FIRST/FOLLOW, construcción de tabla LL(1), y parser no recursivo con recuperación simple de errores.
- Salidas:
 - `tabla_transicion.txt` (entradas de la tabla LL(1))
 - `errores.txt` (errores léxicos y sintácticos)
 - `arbol.dot` (árbol de derivación en Graphviz)
 - `ast.dot` (opcional, actualmente igual a `arbol.dot`)
- GUI (Tkinter) para seleccionar archivos, ejecutar y visualizar resultados.

Manual de uso

3.1. Ejecutar la aplicación

1. Guarde el archivo (por ejemplo) como proyecto2.py.
2. Ejecútelo:
3. `python proyecto2.py`
4. En la ventana:
 - **Archivo de entrada:** seleccione su programa.txt.
 - **Carpeta de salida:** elija dónde se guardarán los reportes.
 - (Opcional) Marque **Generar AST (ast.dot)**.
 - Presione **Analizar**.



3.2. Archivos generados

En la carpeta de salida verá:

- errores.txt → lista de errores léxicos/sintácticos o “Sin errores.”
- tabla_transicion.txt → entradas $M[\text{NoTerminal}, \text{terminal}] = \text{producción}$.
- arbol.dot → árbol de derivación en formato Graphviz.
- ast.dot → igual que arbol.dot por ahora (marcado opcional en la GUI).

Nombre	Fecha de modificación	Tipo	Tamaño
 arbol.dot	23/10/2025 06:23 p. m.	Word.Template.8	2 KB
 ast.dot	23/10/2025 06:23 p. m.	Word.Template.8	2 KB
 errores	23/10/2025 06:23 p. m.	Archivo TXT	1 KB
 expresiones regulares ejercicios	01/08/2025 07:46 a. m.	Chrome HTML Do...	220 KB
 tabla_transicion	23/10/2025 06:23 p. m.	Archivo TXT	3 KB

Gramática soportada

No terminales y producciones

Prog → ClassDecl

ClassDecl → 'class' id '{' MemberList '}'

MemberList → Member MemberList | ε

Member → MethodDecl | VarDecl

VarDecl → Type id ';'

MethodDecl → Type id '(' ParamList ')' Block

ParamList → Param ParamRest | ε

ParamRest → ',' Param ParamRest | ε

Param → Type id

Block → '{' StmtList '}'

StmtList → Stmt StmtList | ε

Stmt → VarDecl | Assign | Return | Call ';'

Assign → id '=' Expr ';'

Return → 'return' Expr ';'

Call → id '(' ArgList ')'

ArgList → Expr ArgRest | ε

ArgRest → ',' Expr ArgRest | ε

Expr → Add RelP

RelP → '<' Add RelP | '>' Add RelP | '==' Add RelP | ε

Add → Term AddP

AddP → '+' Term AddP | '-' Term AddP | ε

Term → Factor TermP

TermP → '*' Factor TermP | '/' Factor TermP | ε

Factor → id | number | '(' Expr ')'

Type → 'int' | 'void'

Palabras clave, símbolos y operadores reconocidos

- **KEYWORDS:** class, int, void, return
- **Símbolos:** { } () ; ,
- **Operadores:** + - * / < > == =

Precedencia y asociatividad (implícita por la gramática)

- Mayor a menor: * / > + - < > == (todas asociativas por la recursividad a la derecha en los sufijos AddP, TermP, RelP).
- Paréntesis forzan agrupación en Factor.

Arquitectura del sistema

Componentes principales

- **Léxico (tokenize)**
Lee el source, emite tokens: List[(tipo, lexema, línea, columna)], maneja comentarios y reporta caracteres ilegales.
- **Construcción LL(1)**
 - compute_FIRST, compute_FOLLOW
 - build_table → Table[(NoTerm, terminal)] = producción + Conflicts (si existen).
- **Parser predictivo no recursivo (parse)**
 - Pila de símbolos y nodos.
 - Selección de producción vía Table.
 - **Recuperación simple**: inserciones y anclaje de errores (<insertado X>, <error: inesperado t>).
 - Construcción del **árbol de derivación** (Node) con to_dot().
- **Salida / Reportes**
 - write_table, write_errors, write_dot, write_ast_dot.
- **GUI (Tkinter)**
 - Selección de archivo y carpeta, ejecución, vistas de tokens/errores/tabla, resumen.

Pruebas del Programa

Prueba 1: Programa correcto

Entrada (programa.txt):

```
class Demo {  
    int x;  
  
    int suma(int a, int b) {  
        return a + b;  
    }  
  
    void main() {  
        x = 2 + 3 * (4 + 5);  
        return;  
    }  
}
```

Resultado esperado:

- errores.txt → “Sin errores.”
- tabla_transicion.txt → contiene todas las reglas aplicadas.
- arbol.dot → se genera correctamente y puede convertirse a imagen con:
dot -Tpng arbol.dot -o arbol.png && start arbol.png

Prueba 2: Error por punto y coma faltante

Entrada:

```
class A {  
    int x  
}
```

Resultado esperado:

- errores.txt muestra un mensaje indicando falta de “;”.
- En arbol.dot aparecerá un nodo <insertado ;> que el parser agrega automáticamente para continuar el análisis.

Prueba 3: Paréntesis no balanceados

Entrada:

```
class A {  
    void m() {  
        x = (1 + 2;  
        return;  
    }  
}
```

Resultado esperado:

- errores.txt muestra error sintáctico cerca de “)”.
• El árbol se genera parcialmente con el nodo <error: inesperado ...>.

Prueba 4: Comentario de bloque no cerrado

Entrada:

```
class A {  
    /* sin cierre  
    int x;  
}
```

Resultado esperado:

- errores.txt indica: “Comentario de bloque no cerrado iniciado en...”.
- Se generan igualmente los demás archivos de salida.

Evidencia de resultados

En el reporte de pruebas se recomienda incluir:

1. Captura de la interfaz (GUI) antes y después de presionar “Analizar”.
2. Fragmento de errores.txt mostrando los mensajes.
3. Vista del arbol.png generado con Graphviz.
4. Fragmento de tabla_transicion.txt con algunas producciones.

Solución del error del archivo .dot

Se corrigió el método to_dot() del código para eliminar un error que rompía el archivo Graphviz.

Antes generaba líneas como:

```
[label="Prog"];
```

Ahora se genera correctamente:

```
[label="Prog"];
```

Además, se agregó el escape automático de comillas y barras invertidas para que el archivo .dot sea válido en cualquier texto o lexema.

Consejos de uso

- Para convertir el árbol:
- `dot -Tpng arbol.dot -o arbol.png && start arbol.png`
- Si dot no se reconoce, reinstala Graphviz y marca la casilla “Add Graphviz to system PATH”.
- Los nodos con <insertado ;> o <error: inesperado X> indican errores recuperados por el parser.
- Puedes abrir los archivos .txt en cualquier editor para revisar la tabla LL(1) o los errores.

Mejoras futuras

- Integrar un visor del árbol dentro del programa (con zoom y desplazamiento).
- Crear un AST más limpio eliminando nodos sintácticos.
- Añadir resaltado de errores en la interfaz y validaciones semánticas básicas.